Lage parts of document based on a similar document by Mike Gill of TU Dublin. AI (Claud) used.

Check vidoe and other links stillexist.

INSTALLING ARDUINO LIBRARIES

TenStar ESP32-S3 Development Board

BIP26 SENSATE-X Programme
TU Dublin - April 2026

Based on setup guide by Mike Gill, TU Dublin
Version 1.3 - Updated with GPS/GNSS Libraries

TABLE OF CONTENTS

---

1. WHAT ARE LIBRARIES?

Simple Explanation:

A **library** is a collection of pre-written code that performs specific tasks, so you don't have to write everything from scratch.

Think of it like this:

Imagine you want to bake a cake. Instead of:

- Growing wheat to make flour
- Raising chickens to get eggs
- Processing sugar cane

You simply go to the shop and **buy the ingredients** (flour, eggs, sugar).

Arduino libraries work the same way:

- Instead of writing hundreds of lines of code to control a display
- You **install a display library**
- Then use simple commands like tft.print("Hello") to show text

---

Real Example:

Without a library, to display "Hello" on the TFT screen, you'd need to write code to:

1. Initialize SPI communication protocol
2. Send low-level commands to the ST7789 chip
3. Set pixel coordinates
4. Convert text characters to pixel patterns
5. Send pixel data to the screen
6. Handle timing and synchronization

≈ 500+ lines of complex code!

With the Adafruit_ST7789 library:

```
tft.print("Hello");
```

Just 1 line! ✓

What's Inside a Library?

A library typically contains:

1. Functions - Ready-made commands you can use

- Example: tft.fillScreen(BLACK) - fills screen with black color

2. Definitions - Pre-configured settings

- Example: ST77XX_WHITE - defines the color white

3. Examples - Sample code showing how to use the library

- Example: "How to draw shapes on the display"

4. Documentation - Instructions on what each function does

- Example: "fillScreen(color) - fills entire screen with specified color"

---

2. WHY DO WE NEED LIBRARIES?

Key Benefits:

1. Save Time

- Don't reinvent the wheel
- Focus on your project, not low-level details
- Hours/days of work reduced to minutes

2. Reliability

- Libraries are tested by thousands of users
- Bugs already found and fixed
- More reliable than code you write from scratch

3. Compatibility

- Libraries work across different Arduino boards
- Handle hardware differences automatically
- Manufacturer-supported for specific components

4. Easy Updates

- Library developers improve code over time
- You get bug fixes and new features automatically
- Just update the library when new versions available

5. Community Support

- Popular libraries have extensive documentation

- Online forums help if you get stuck
- Example code shows best practices

For BIP26 Project:

You'll use libraries to control:

Hardware Component	What It Does	Library Used
TFT Display	Shows text and graphics	Adafruit_ST7789
Graphics	Draws shapes, text, colors	Adafruit_GFX
RGB LED	Multicolor light (WS2812)	Adafruit_NeoPixel
BMP280 Sensor	Measures temperature & pressure	Adafruit_BMP280
QMI8658 Sensor	Gyroscope & accelerometer	SensorLib
GPS Data Parsing	Converts NMEA to usable data	TinyGPSPlus

Without these libraries, the project would be impossible in one week!

---

3. HOW LIBRARIES WORK

The Include Statement:

At the top of your Arduino code, you'll see:

#include <Adafruit_ST7789.h>

This line:

1. Tells Arduino IDE "I want to use the ST7789 library"
2. Arduino loads all the pre-written code from that library
3. Makes library functions available in your program

It's like saying: "I need the TFT display toolkit - please make it available to me."

Using Library Functions:

Once included, you can use the library's functions:

```
#include <Adafruit_ST7789.h>
```

```
Adafruit_ST7789 tft = Adafruit_ST7789(7, 39, 40);  // Create TFT
object

void setup() {
  tft.init(135, 240);           // Initialize display
  tft.fillScreen(ST77XX_BLACK); // Fill with black
  tft.setTextColor(ST77XX_WHITE); // Set text color to white
  tft.print("Hello BIP26!");    // Display text
}
```

Each function (init, fillScreen, setTextColor, print) **comes from the library** - you didn't have to write any of that code!

Where Libraries Are Stored:

After installation, libraries are stored in:

Windows:

C:\Users\[YourName]\Documents\Arduino\libraries\

Mac:

~/Documents/Arduino/libraries/

Each library gets its own folder:

```
Arduino/
  └── libraries/
      ├── Adafruit_NeoPixel/
      ├── Adafruit_GFX/
      ├── Adafruit_ST7789/
      ├── Adafruit_BMP280/
      ├── SensorLib/
      └── TinyGPSPlus/
```

You rarely need to look in these folders - Arduino IDE manages everything automatically.

---

4. INSTALLING LIBRARIES - OVERVIEW

Three Ways to Install Libraries:

Method 1: Library Manager (Recommended - Easiest!) ✓

- Built into Arduino IDE
- One-click installation

- Automatic updates
- **This is what we'll use**

Method 2: ZIP File

1. Download .zip file from website
2. Sketch → Include Library → Add .ZIP Library
3. Useful for custom/unreleased libraries

Method 3: Manual Installation

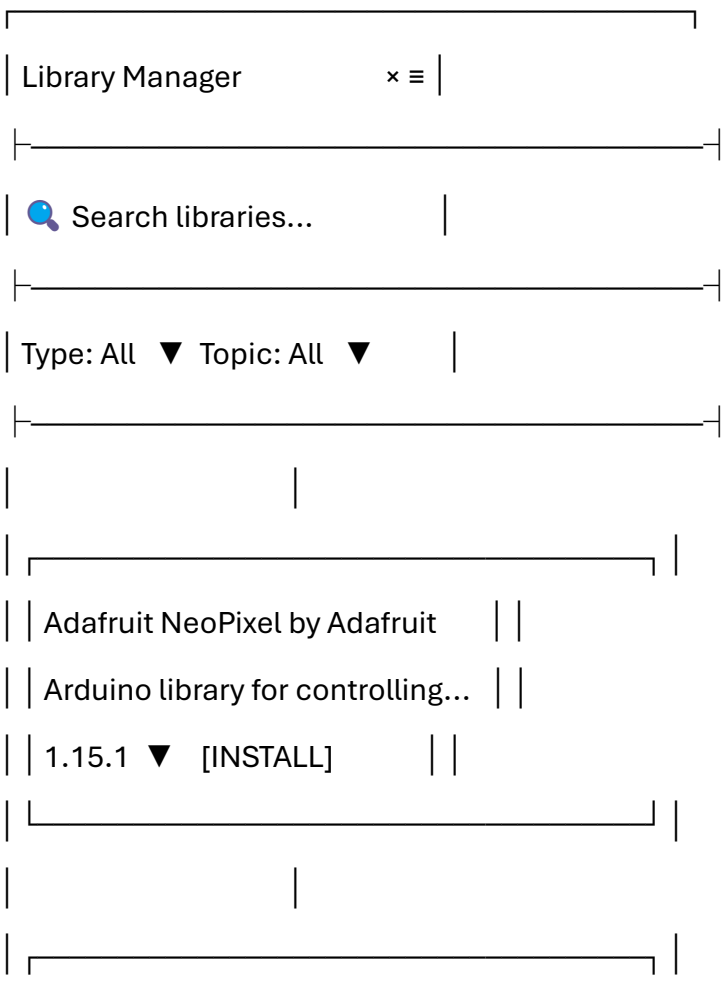
1. Download library folder
2. Copy to Arduino/libraries/ folder
3. Restart Arduino IDE
4. Only for advanced users

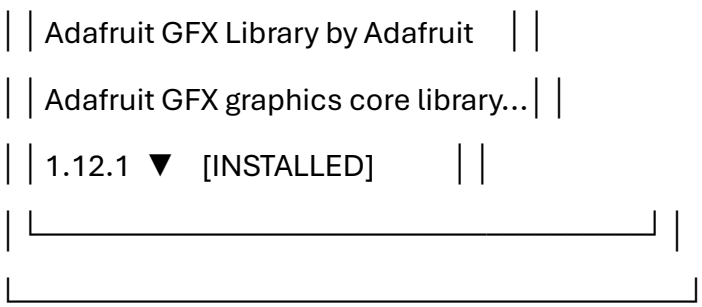
For BIP26, we'll use Method 1 (Library Manager) - it's simple and reliable.

Library Manager Interface:

Location: Tools → Manage Libraries...

What you'll see:





Key elements:

1. Search box: Type library name
2. Version dropdown: Usually keep default (latest)
3. INSTALL button: Click to install
4. INSTALL ALL: When library has dependencies

---

5. STEP-BY-STEP LIBRARY INSTALLATION

Prerequisites:

Before installing libraries, ensure:

1. [ ] Arduino IDE is installed and running
2. [ ] ESP32 board support is installed (ESP32S3 Dev Module)
3. [ ] TenStar board is connected via USB (optional, but good to verify)
4. [ ] You have stable internet connection (libraries download from online)

General Installation Procedure:

Follow these steps for EACH library:

Step 1: Open Library Manager

- Click **Tools → Manage Libraries...**
- Library Manager window opens

Step 2: Search for Library

- In search box, type the library name (e.g., "Adafruit_NeoPixel")
- Wait for results to appear

Step 3: Identify Correct Library

- Look for exact name match
- Verify author (usually "Adafruit" or as specified below)
- Check description matches what you need

Step 4: Install Library

- Click **INSTALL** button
- If prompted about dependencies, click **INSTALL ALL**
- Wait for "INSTALLED" to appear

Step 5: Verify Installation

- "INSTALLED" appears next to library name
- Green checkmark may appear

Step 6: Close Library Manager

- Click X or close window
- Library is now ready to use!

---

6. REQUIRED LIBRARIES FOR TENSTAR BOARD

Complete Library List:

You need to install **6 libraries** for the TenStar board to work with all its onboard components and GPS module.

LIBRARY 1: Adafruit NeoPixel

What it controls: WS2812 RGB LED (the multicolor light on your board)

Installation:

1. **Open Library Manager:** Tools → Manage Libraries...
2. **Search:** Type Adafruit_NeoPixel in search box
3. **Find library:**
    - **Name:** Adafruit NeoPixel
    - **Author:** Adafruit
    - **Description:** "Arduino library for controlling single-wire-based LED pixels and strip..."
4. **Install:**
    - Click **INSTALL** button
    - Current version (as of April 2026): ~1.15.x

- No dependencies - installs quickly

5. **Verify:**
    - Button changes to **INSTALLED**
    - Library is ready!

What you can do with it:

- Change LED color
- Create rainbow effects
- Adjust brightness
- Animate colors

LIBRARY 2: Adafruit GFX Library

What it controls: Graphics functions (drawing shapes, text, etc.)

Installation:

1. **Search:** Type Adafruit_GFX in Library Manager
2. **Find library:**
    - **Name:** Adafruit GFX Library
    - **Author:** Adafruit
    - **Description:** "Adafruit GFX graphics core library, this is the 'core' class..."
3. **Install:**
    - Click **INSTALL** button
    - **IMPORTANT:** When prompted "Install library dependencies"
    - Click **INSTALL ALL** (installs Adafruit_BusIO and other dependencies)
    - Current version: ~1.12.x
4. **Verify:**
    - Main library shows **INSTALLED**
    - Dependencies also show **INSTALLED**

What you can do with it:

- Draw lines, rectangles, circles

- Display text in different sizes
- Create graphics and animations
- Set colors and fonts

LIBRARY 3: Adafruit ST7735 and ST7789 Library

The ST7735 and ST7789 refer to popular TFT (Thin Film Transistor) LCD display controller drivers manufactured by the Taiwanese company Sitronix Technology Corporation. . They are integrated circuit chips used to drive small, colorful, high-speed graphical displays commonly used in embedded systems, DIY electronics projects, and prototyping with microcontrollers like Arduino, ESP32, and Raspberry Pi. The ST7789 is used on the tenstart board.

What it controls: TFT Display (the 1.14" color screen on your board)

Installation:

1. **Search:** Type Adafruit_ST7789 in Library Manager
2. **Find library:**
    - **Name:** Adafruit ST7735 and ST7789 Library
    - **Author:** Adafruit
    - **Description:** "This is a library for the Adafruit ST7735 and ST7789 SPI displays..."
    - **Note:** One library supports BOTH ST7735 and ST7789 displays
3. **Install:**
    - Click **INSTALL** button
    - When prompted about dependencies, click **INSTALL ALL**
    - Installs: Adafruit_BusIO, SD, SPI (if not already installed)
    - Current version: ~1.11.x
4. **Verify:**
    - Shows **INSTALLED**
    - All dependencies also installed

What you can do with it:

- Initialize the TFT display
- Clear screen, fill with colors
- Display text and graphics
- Control screen orientation
- Adjust backlight

LIBRARY 4: Adafruit BMP280 Library

The BMP280 (Bosch Sensortec GmbH,Reutlingen) is a digital barometric pressure and temperature sensor , It used MEMS (Micro-Electro-Mechanical Systems) technology.

What it controls: BMP280 sensor (measures temperature and atmospheric pressure)

Installation:

1. **Search:** Type Adafruit_BMP280 in Library Manager
2. **Find library:**
    - **Name:** Adafruit BMP280 Library
    - **Author:** Adafruit
    - **Description:** "Arduino library for BMP280 sensors. Arduino library for BMP280..."
3. **Install:**
    - Click **INSTALL** button
    - When prompted, click **INSTALL ALL**
    - Installs dependencies: Adafruit_BusIO, Adafruit Unified Sensor
    - Current version: ~2.6.x
4. **Verify:**
    - Main library and dependencies show **INSTALLED**

What you can do with it:

- Read temperature (°C or °F)
- Read atmospheric pressure (hPa)
- Calculate approximate altitude

- Monitor environmental conditions

LIBRARY 5: SensorLib (QMI8658)

Manufactured by Qst Corporation of Shanghai, The QMI8658C is a 6-axis IMU (Inertial Measurement Unit). IMU = Inertial Measurement Unit

1. Device that measures motion and orientation
2. Uses internal sensors (not external references like GPS)
3. Detects changes in movement and rotation

What it controls: QMI8658 sensor (6-axis gyroscope and accelerometer)

Installation:

1. **Search:** Type QMI8658 in Library Manager
2. **Find library:**
    - **Name:** SensorLib
    - **Author:** Lewis He (or lewisxhe)
    - **Description:** "Commonly used I2C, SPI device multi-platform libraries Support: QMC6310, QMI8658, PCF8563, PCF8..."
3. **Install:**
    - Click **INSTALL** button
    - Usually no dependencies
    - Current version: ~0.3.x
4. **Verify:**
    - Shows **INSTALLED**

What you can do with it:

1. Read acceleration (X, Y, Z axes)
2. Read rotation rate (gyroscope)
3. Detect motion and orientation
4. Measure tilt angles
5. Detect impacts or vibration

---

7. GPS/GNSS LIBRARIES

Why You Need a GPS Library:

Your Waveshare LC29H GPS module sends data in **NMEA format** - text sentences that look like this:

$GNGLL,5317.377320,N,00621.258036,W,084318.000,A,A*5E

Without a GPS library, you'd need to:

- Split the sentence by commas manually
- Convert coordinates from DDMM.MMMM format to decimal degrees
- Validate checksums
- Handle multiple sentence types
- Track which data is valid

≈ 200+ lines of complex string parsing code!

With TinyGPSPlus library:

double latitude = gps.location.lat(); // Just 1 line!

LIBRARY 6: TinyGPSPlus ⭐ ESSENTIAL FOR GPS WORK

What it controls: NMEA sentence parsing (converts GPS text data into usable numbers)

Why it's the #1 choice:

1. ✅ **Most popular GPS library** - millions of downloads worldwide
2. ✅ **Beginner-friendly** - very easy to learn and use
3. ✅ **Comprehensive** - extracts all useful GPS data automatically
4. ✅ **Lightweight** - uses minimal memory
5. ✅ **Well-documented** - excellent examples included
6. ✅ **Universal** - works with ALL GPS modules (not brand-specific)
7. ✅ **Industry-standard** - used in commercial products

Installation:

1. **Search:** Type TinyGPSPlus in Library Manager
2. **Find library:**
    - **Name:** TinyGPSPlus
    - **Author:** Mikal Hart
    - **Description:** "A compact Arduino NMEA (GPS) parsing library"
3. **Install:**
    - Click **INSTALL** button

- No dependencies required
- Current version: ~1.1.x

4. **Verify:**
    - Shows **INSTALLED**
    - Ready to use!

What you can extract with TinyGPSPlus:

Position data:

1. Latitude in decimal degrees
2. Longitude in decimal degrees
3. Altitude above sea level
4. Validity check (is position accurate?)

Motion data:

1. Speed (in km/h, mph, knots, m/s)
2. Course/heading (direction of travel in degrees)

Time and date:

1. UTC date (day, month, year)
2. UTC time (hour, minute, second, centisecond)

Satellite information:

- Number of satellites being tracked
- HDOP (Horizonal Dilution of Precision. accuracy indicator)

Diagnostics:

3. Count of valid sentences received
4. Count of failed checksums (corrupted data)
5. Characters processed

This library is ESSENTIAL for BIP26 - it saves you hours of work and lets you focus on OSNMA authentication instead of basic NMEA parsing!

---

8. TINYGPSPLUS TUTORIAL

What is TinyGPSPlus?

TinyGPSPlus is an Arduino library that automatically converts GPS NMEA sentences into easy-to-use data. Instead of manually parsing complex text strings, you get simple functions that give you exactly what you need.

Created by: Mikal Hart
License: LGPL (free to use)
Status: Mature, stable, widely-used

Basic Concept:

1. GPS sends NMEA sentences:

$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47

$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A

2. You feed these to TinyGPSPlus:

```
while (Serial1.available()) {
  gps.encode(Serial1.read());  // Feed each character
}
```

3. TinyGPSPlus parses everything automatically:

6. Extracts latitude: 48.117300°
7. Extracts longitude: 11.516667°
8. Extracts satellites: 8
9. Validates checksums
10. Converts formats

4. You get simple access to data:

```
double lat = gps.location.lat();      // 48.117300
double lng = gps.location.lng();      // 11.516667
int sats = gps.satellites.value();    // 8
```

Complete Working Example:

```
/*
 * TinyGPSPlus Basic Example
 * Reads GPS data and displays position, satellites, speed
 */

#include <TinyGPSPlus.h>

// Create TinyGPSPlus object
TinyGPSPlus gps;

// GPS connected to Serial1 (GPIO17/18 on TenStar)
#define GPS_RX 17
#define GPS_TX 18
```

```
void setup() {
  // USB Serial for debugging
  Serial.begin(115200);

  // GPS Serial at 115200 baud (your LC29H configuration)
  Serial1.begin(115200, SERIAL_8N1, GPS_RX, GPS_TX);

  Serial.println("TinyGPSPlus Example");
  Serial.println("Waiting for GPS data...");
}

void loop() {
  // Feed all available GPS data to TinyGPSPlus
  while (Serial1.available() > 0) {
    char c = Serial1.read();
    gps.encode(c);  // Parse each character
  }

  // Display parsed data every 2 seconds
  static unsigned long lastDisplay = 0;
  if (millis() - lastDisplay > 2000) {
    displayGPSInfo();
    lastDisplay = millis();
  }
}

void displayGPSInfo() {
  Serial.println("==== GPS DATA ====");

  // Location
  if (gps.location.isValid()) {
    Serial.print("Latitude: ");
    Serial.println(gps.location.lat(), 6);  // 6 decimal places
    Serial.print("Longitude: ");
    Serial.println(gps.location.lng(), 6);
  } else {
    Serial.println("Location: INVALID (waiting for fix)");
  }

  // Altitude
  if (gps.altitude.isValid()) {
    Serial.print("Altitude: ");
    Serial.print(gps.altitude.meters());
    Serial.println(" meters");
  }

  // Speed
  if (gps.speed.isValid()) {
    Serial.print("Speed: ");
```

```
    Serial.print(gps.speed.kmph());
    Serial.println(" km/h");
  }

  // Satellites
  if (gps.satellites.isValid()) {
    Serial.print("Satellites: ");
    Serial.println(gps.satellites.value());
  }

  // Date and Time (UTC)
  if (gps.date.isValid() && gps.time.isValid()) {
    Serial.print("Date/Time: ");
    Serial.print(gps.date.day());
    Serial.print("/");
    Serial.print(gps.date.month());
    Serial.print("/");
    Serial.print(gps.date.year());
    Serial.print(" ");
    Serial.print(gps.time.hour());
    Serial.print(":");
    Serial.print(gps.time.minute());
    Serial.print(":");
    Serial.println(gps.time.second());
  }

  // Accuracy (HDOP - lower is better)
  if (gps.hdop.isValid()) {
    Serial.print("HDOP (accuracy): ");
    Serial.println(gps.hdop.hdop());
  }

  Serial.println("==================");
}
```

What this code does:

1. Continuously reads GPS data from Serial1
2. Feeds each character to gps.encode()
3. TinyGPSPlus parses NMEA sentences automatically
4. Every 2 seconds, displays all parsed data
5. Shows "INVALID" if GPS doesn't have a fix yet

Key TinyGPSPlus Objects and Functions:

Location (Position):

```
gps.location.lat()          // Latitude in decimal degrees (-90 to
+90)
```

```
gps.location.lng()           // Longitude in decimal degrees (-180
to +180)
gps.location.isValid()       // true if position data is valid
gps.location.isUpdated()     // true if position updated since last
check
gps.location.age()           // milliseconds since last update
```

Altitude:

```
gps.altitude.meters()        // Altitude in meters
gps.altitude.feet()          // Altitude in feet
gps.altitude.kilometers()    // Altitude in kilometers
gps.altitude.miles()         // Altitude in miles
gps.altitude.isValid()       // true if altitude data valid
```

Speed:

```
gps.speed.kmph()            // Speed in kilometers per hour
gps.speed.mph()             // Speed in miles per hour
gps.speed.mps()             // Speed in meters per second
gps.speed.knots()           // Speed in knots (nautical miles per
hour)
gps.speed.isValid()         // true if speed data valid
```

Course (Direction of Travel):

```
gps.course.deg()            // Course in degrees (0-360)
gps.course.isValid()        // true if course data valid
```

Date (UTC):

```
gps.date.day()              // Day (1-31)
gps.date.month()            // Month (1-12)
gps.date.year()             // Year (4 digits, e.g., 2026)
gps.date.isValid()          // true if date valid
```

Time (UTC):

```
gps.time.hour()             // Hour (0-23)
gps.time.minute()           // Minute (0-59)
gps.time.second()           // Second (0-59)
gps.time.centisecond()      // Centisecond (0-99)
gps.time.isValid()          // true if time valid
```

Satellites:

```
gps.satellites.value()      // Number of satellites in use
gps.satellites.isValid()    // true if satellite count valid
```

Accuracy (HDOP):

```
gps.hdop.hdop()             // Horizontal Dilution of Precision
                            // Lower = better (< 2 is excellent)
gps.hdop.isValid()          // true if HDOP valid
```

Diagnostics:

```
gps.charsProcessed()        // Total characters processed
gps.sentencesWithFix()      // Sentences with valid fix
```

```
gps.failedChecksum()        // Sentences with bad checksum
gps.passedChecksum()        // Sentences with good checksum
```

Understanding GPS Speed Units:

What is a "knot"?

A **knot** is a unit of speed used in maritime and aviation navigation.

Definition:

- **1 knot = 1 nautical mile per hour**
- **1 nautical mile = 1.852 kilometers = 1.15078 statute miles**

Why "knot"?

The name comes from old sailing ships:

1. Sailors would throw a rope overboard with knots tied at regular intervals
2. They counted how many knots passed through their hands in a fixed time
3. More knots = faster speed
4. Hence: "making 10 knots" = traveling at 10 nautical miles per hour

Why use knots instead of km/h or mph?

1. **Nautical miles match Earth's geometry:**
    - 1 nautical mile = 1 minute of latitude (1/60th of a degree)
    - Makes navigation calculations simpler on charts
    - Directly relates to degrees on a map
2. **Universal in marine/aviation:**
    - International standard for ships and aircraft
    - Charts and maps use nautical miles
    - Air traffic control uses knots

Conversion examples:

Knots	km/h	mph	m/s
1	1.852	1.151	0.514
5	9.26	5.75	2.57
10	18.52	11.51	5.14

Knots	km/h	mph	m/s
20	37.04	23.02	10.29
100	185.2	115.1	51.4

TinyGPSPlus makes conversion easy:

```
// GPS reports speed in knots in NMEA sentences
// TinyGPSPlus converts automatically:

double speedKnots = gps.speed.knots();  // Original (nautical)
double speedKmh = gps.speed.kmph();     // Metric
double speedMph = gps.speed.mph();      // Imperial
double speedMps = gps.speed.mps();      // Scientific

// Use whichever unit suits your application!
```

For BIP26: Most students will use gps.speed.kmph() since we're in Europe, but it's good to know about knots for maritime/aviation contexts!

Common Usage Patterns:

Pattern 1: Check if GPS has a fix before using data

```
if (gps.location.isValid()) {
  // Safe to use position data
  double lat = gps.location.lat();
  double lng = gps.location.lng();
} else {
  // GPS doesn't have a fix yet
  Serial.println("Waiting for GPS fix...");
}
```

Pattern 2: Check if data was recently updated

```
if (gps.location.isUpdated()) {
  // Position just changed - display new coordinates
  displayPosition();
}
```

Pattern 3: Calculate distance between two points

```
// TinyGPSPlus includes distance calculation!
double distanceMeters = TinyGPSPlus::distanceBetween(
  lat1, lng1,  // Point 1
  lat2, lng2   // Point 2
);

double courseTo = TinyGPSPlus::courseTo(
  lat1, lng1,  // From point
```

```
  lat2, lng2   // To point
);
```

Pattern 4: Display on TFT screen

```
if (gps.location.isValid()) {
  tft.fillScreen(ST77XX_BLACK);

  tft.setCursor(5, 10);
  tft.print("Lat: ");
  tft.println(gps.location.lat(), 6);

  tft.setCursor(5, 30);
  tft.print("Lon: ");
  tft.println(gps.location.lng(), 6);


  tft.setCursor(5, 50);
  tft.print("Sats: ");
  tft.println(gps.satellites.value());


  tft.setCursor(5, 70);
  tft.print("Speed: ");
  tft.print(gps.speed.kmph(), 1);
  tft.println(" km/h");
}
```

Troubleshooting TinyGPSPlus:

Problem: gps.location.isValid() always returns false

Causes:

1. GPS doesn't have satellite fix yet (most common)
2. GPS not sending NMEA data
3. Wrong baud rate

Solutions:

- Wait 30-60 seconds for GPS to acquire satellites
- Move near window or outdoors (GPS needs sky view)
- Check Serial Monitor - are you seeing NMEA sentences?
- Verify baud rate matches GPS (115200 for your LC29H)

Problem: gps.charsProcessed() returns 0

Causes:

1. No data coming from GPS
2. GPS not powered
3. TX/RX wires swapped or disconnected

Solutions:

- Check GPS power LED is lit
- Verify wire connections (GPS TX → ESP32 GPIO17)
- Test with raw Serial1.read() to see if any data arrives

Problem: gps.failedChecksum() increasing

Causes:

1. Electrical interference
2. Loose wire connection
3. Wrong baud rate

Solutions:

- Check all wires firmly connected
- Verify baud rate (115200 for LC29H)
- Move away from sources of interference

Problem: Data seems old (gps.location.age() is large)

Causes:

1. GPS update rate is slow
2. Not calling gps.encode() frequently enough

Solutions:

- Call gps.encode() in loop() without delays
- Check GPS is still sending data
- GPS update rate is typically 1 Hz (once per second) - this is normal

---

Learning Resources for TinyGPSPlus:

Official Documentation:

- GitHub Repository: https://github.com/mikalhart/TinyGPSPlus
- Documentation: http://arduiniana.org/libraries/tinygpsplus/
- Examples included with library (File → Examples → TinyGPSPlus)

YouTube Video Tutorials:

1. "Arduino GPS Tutorial - Using TinyGPS++ Library"

- Channel: DroneBot Workshop
- URL: https://www.youtube.com/watch?v=mVTL_SWCeS0
- Duration: ~20 minutes
- **Excellent beginner tutorial!** Step-by-step setup and explanation
- Shows hardware connection and code walkthrough

2. "GPS Module with Arduino - Complete Guide"

- Channel: Last Minute Engineers
- URL: https://www.youtube.com/watch?v=hQw9U_TbsPs
- Duration: ~15 minutes
- Good overview of GPS basics and TinyGPSPlus usage

3. "NEO-6M GPS Module Arduino Tutorial"

- Channel: Electronoobs
- URL: https://www.youtube.com/watch?v=P_N-GhZ4-NE
- Duration: ~18 minutes
- Hardware-focused, shows wiring and TinyGPSPlus implementation

4. "Arduino GPS Speedometer using TinyGPS++"

- Channel: Science Projects
- Duration: ~12 minutes
- Shows practical project using speed functions

Written Tutorials:

1. DroneBot Workshop Article:

- URL: https://dronebotworkshop.com/neo-6m-gps/
- Very comprehensive written guide
- Covers NMEA format, wiring, and TinyGPSPlus code
- Excellent diagrams and explanations

2. Last Minute Engineers:

- URL: https://lastminuteengineers.com/neo6m-gps-arduino-tutorial/
- Step-by-step tutorial with clear images
- Good explanation of GPS basics
- Multiple code examples

3. Arduino Project Hub:

- Search: "TinyGPSPlus" on create.arduino.cc
- Multiple community projects using the library
- Real-world applications and examples

4. Random Nerd Tutorials:

- URL: https://randomnerdtutorials.com/
- Search for "GPS" - several ESP32 GPS tutorials
- Uses TinyGPSPlus with ESP32 boards

Example Code Snippets:

Included with Library: When you install TinyGPSPlus, Arduino IDE includes several examples:

1. **File → Examples → TinyGPSPlus → FullExample**
    - Comprehensive example showing all features
    - Best starting point for learning
2. **File → Examples → TinyGPSPlus → SimpleExample**
    - Minimal code to get started
    - Just displays lat/long and satellites
3. **File → Examples → TinyGPSPlus → DeviceExample**
    - Shows diagnostic information

- Good for troubleshooting

Recommended Learning Path:

Week 1:

1. Watch DroneBot Workshop video (20 min)
2. Read DroneBot Workshop article
3. Run SimpleExample from library

Week 2: 4. Run FullExample from library 5. Modify example to display on TFT 6. Add speed and course display

Week 3: 7. Calculate distance between points 8. Create complete GPS data logger 9. Integrate with OSNMA authentication (BIP26 goal)

---

9. VERIFICATION AND TESTING

Check All Libraries Installed:

After installing all 6 libraries, verify installation:

Method 1: Via Library Manager

1. Open Library Manager (Tools → Manage Libraries...)
2. Search for each library name
3. Verify each shows **INSTALLED**

Checklist:

- [ ] Adafruit NeoPixel - INSTALLED ✓
- [ ] Adafruit GFX Library - INSTALLED ✓
- [ ] Adafruit ST7735 and ST7789 Library - INSTALLED ✓
- [ ] Adafruit BMP280 Library - INSTALLED ✓
- [ ] SensorLib - INSTALLED ✓
- [ ] **TinyGPSPlus - INSTALLED ✓** (NEW!)

Method 2: Via Include Statement

1. Open new sketch in Arduino IDE
2. Type: #include <TinyGPSPlus.h>
3. If no red underline appears → Library installed correctly ✓

4. If red underline → Library not found, reinstall

Test with Example Code:

Test TinyGPSPlus installation:

1. **File → Examples → TinyGPSPlus → SimpleExample**
2. Update GPS serial configuration for your board:
3. // Change this line:// #define ss Serial1// To this for TenStar:Serial1.begin(115200, SERIAL_8N1, 17, 18);
4. Upload to board
5. Open Serial Monitor (115200 baud)
6. You should see GPS data displayed

If successful:

Latitude Longitude Satellites HDOP Date Time

deg deg Count n/a

53.289555 -6.354301 24 1.20 04/03/2026 08:43:18

If you see this, TinyGPSPlus is working perfectly! ✓

Test with Starter Sketch:

The ultimate hardware test is Mike Gill's starter sketch:

1. Copy starter sketch code (from Mike Gill's guide page 4-7)
2. Paste into Arduino IDE
3. Click **Verify** button (checkmark icon)
4. **If it compiles successfully** → All hardware libraries working! ✓
5. **If you get errors** → See troubleshooting section below

Example successful compilation:

Sketch uses 234,156 bytes (17%) of program storage space.

Global variables use 15,432 bytes (4%) of dynamic memory.

---

10. TROUBLESHOOTING

Problem 1: "Library Not Found" Error

Symptom:

fatal error: TinyGPSPlus.h: No such file or directory

Solutions:

A) Library not installed:

- Open Library Manager
- Search for "TinyGPSPlus"
- Click INSTALL
- Restart Arduino IDE

B) Wrong library name in code:

- Check spelling: TinyGPSPlus.h not TinyGPS.h
- Note: There's an older library called "TinyGPS" (without Plus) - make sure you have TinyGPS**Plus**

C) Restart Arduino IDE:

- Close Arduino IDE completely
- Reopen Arduino IDE
- Try compiling again

Problem 2: "Multiple Libraries Found"

Symptom:

Multiple libraries were found for "TinyGPSPlus.h"

Cause: You installed the library more than once or have conflicting versions

Solution:

1. Close Arduino IDE
2. Navigate to Arduino/libraries/ folder
3. Look for duplicate folders (e.g., TinyGPSPlus and TinyGPSPlus-master)
4. Delete duplicate/older version

5. Keep only one copy
6. Restart Arduino IDE

Problem 3: TinyGPS vs TinyGPSPlus Confusion

Important: There are TWO different libraries:

TinyGPS (old, obsolete)

- Original version by Mikal Hart
- Last updated ~2013
- Limited features
- **Don't use this one!**

TinyGPSPlus (current, maintained)

- Updated version by same author
- Actively maintained
- Many more features
- **Use this one!** ✓

If you accidentally installed TinyGPS:

1. Library Manager → Search "TinyGPS"
2. Find "TinyGPS" (not Plus) → Click REMOVE
3. Search "TinyGPSPlus" → Click INSTALL

Problem 4: GPS Data Not Parsing

Symptom:

gps.location.isValid() // Always returns false

gps.charsProcessed() // Returns 0

Causes:

1. Not feeding data to gps.encode()
2. GPS not sending NMEA data
3. Wrong baud rate

Solutions:

Check basic GPS reception:

```
void loop() {
 // Print raw GPS data
 if (Serial1.available()) {
  Serial.write(Serial1.read());
 }
}
```

If you see NMEA sentences → GPS works, problem is with TinyGPSPlus usage

If you see nothing → GPS connection problem (wires, baud rate, power)

Other Common Problems:

See Section 8 troubleshooting for more GPS-specific issues.

---

11. LIBRARY VERSION INFORMATION

Version Numbers Explained:

Libraries use **semantic versioning**: Major.Minor.Patch

Example: TinyGPSPlus **1.1.0**

- **1** = Major version (big changes, may break old code)
- **1** = Minor version (new features, backwards compatible)
- **0** = Patch version (bug fixes)

General rule: Install the **latest version** unless instructed otherwise.

Expected Versions (April 2026):

Note: These version numbers may be different when you install. **This is normal!** Always use the latest version available in Library Manager.

Library Name	Expected Version	Notes
Adafruit NeoPixel	1.15.x or newer	Very stable library

Library Name	Expected Version	Notes
Adafruit GFX	1.12.x or newer	Core graphics library
Adafruit ST7735/ST7789	1.11.x or newer	Display driver
Adafruit BMP280	2.6.x or newer	Sensor library
SensorLib	0.3.x or newer	QMI8658 support
TinyGPSPlus	1.1.x or newer	GPS NMEA parser

Dependencies (auto-installed):

- Adafruit BusIO: ~1.16.x
- Adafruit Unified Sensor: ~1.1.x
- SD: ~1.2.x (built-in Arduino library)
- SPI: Built-in Arduino library
- Wire: Built-in Arduino library

Checking Installed Versions:

Method 1: Library Manager

1. Tools → Manage Libraries...
2. Search for library name
3. Version shown next to library name
4. If update available: "UPDATE" button appears

Method 2: File System

1. Navigate to: Documents/Arduino/libraries/
2. Open library folder (e.g., TinyGPSPlus)
3. Open library.properties file with text editor
4. Look for line: version=1.1.0

Should I Update Libraries?

General guidance:

YES - Update if:

- ✓ New version fixes bugs you've encountered
- ✓ New version adds features you need
- ✓ Your code isn't working and update might fix it
- ✓ It's been 6+ months since last update

NO - Don't update if:

- ✗ Your project is working perfectly
- ✗ You're in the middle of important work
- ✗ Update notes mention "breaking changes"
- ✗ You're working on assessed coursework (keep stable version)

For BIP26: Use whatever version Library Manager offers when you install. It will work correctly.

---

12. ADDITIONAL RESOURCES

Arduino Library Documentation:

- Arduino Libraries Guide: https://docs.arduino.cc/learn/starting-guide/libraries
- Arduino Library Manager: https://docs.arduino.cc/software/ide-v2/tutorials/ide-v2-installing-a-library

Adafruit Learning:

- Adafruit Learning System: https://learn.adafruit.com
- Library tutorials and examples
- Hardware guides and datasheets

TinyGPSPlus Resources:

- GitHub: https://github.com/mikalhart/TinyGPSPlus
- Documentation: http://arduiniana.org/libraries/tinygpsplus/
- DroneBot Workshop Tutorial: https://dronebotworkshop.com/neo-6m-gps/
- Last Minute Engineers: https://lastminuteengineers.com/neo6m-gps-arduino-tutorial/

GPS/GNSS Learning:

- How GPS Works: https://www.gps.gov/systems/gps/
- NMEA Protocol Explained: https://www.nmea.org/
- Understanding HDOP: https://en.wikipedia.org/wiki/Dilution_of_precision_(navigation)

Getting Help:

- Arduino Forum: https://forum.arduino.cc
- Adafruit Forum: https://forums.adafruit.com
- BIP26 instructors and team members

SUMMARY

Quick Reference - Installation Steps:

For each of the 6 required libraries:

1. Tools → Manage Libraries...
2. Search for library name
3. Click INSTALL (or INSTALL ALL if prompted)
4. Wait for INSTALLED to appear
5. Close Library Manager

Libraries to install:

1. Adafruit NeoPixel
2. Adafruit GFX Library
3. Adafruit ST7735 and ST7789 Library
4. Adafruit BMP280 Library
5. SensorLib (for QMI8658)
6. **TinyGPSPlus (for GPS parsing) ⭐ ESSENTIAL!**

Total time: 5-10 minutes

What You've Learned:

✓ What libraries are and why they're useful
✓ How to use Arduino Library Manager
✓ How to install libraries with dependencies
✓ How to verify libraries are installed correctly
✓ How to troubleshoot common library problems
✓ Understanding library version numbers
✓ **How to parse GPS NMEA data with TinyGPSPlus**
✓ **What knots are and why they're used in navigation**

Next Steps:

After installing libraries:

1. Test with Mike Gill's starter sketch (hardware)
2. Test TinyGPSPlus with SimpleExample (GPS)
3. Verify all onboard components work (LED, display, sensors)
4. Create GPS display on TFT screen
5. Begin BIP26 OSNMA authentication work!

Good luck with your BIP26 project!

END OF DOCUMENT

Prepared by: Gerard Stockil, TU Dublin Tallaght Campus
Based on: Mike Gill's TenStar Setup Guide Ver 1.1
For: BIP26 SENSATE-X Programme 2026
Date: April 2026
Version: 1.3 (Updated with TinyGPSPlus and GPS tutorial)

This complete updated document now includes:

✅ TinyGPSPlus as Library #6
✅ Comprehensive TinyGPSPlus tutorial (Section 8)
✅ Explanation of what a "knot" is in navigation
✅ Links to YouTube tutorials and written resources

- ✅ Complete code examples
- ✅ All TinyGPSPlus objects and functions documented
- ✅ Common usage patterns
- ✅ Troubleshooting GPS-specific issues
- ✅ Updated checklist with 6 libraries

Copy this into Word and save as "Installing_Arduino_Libraries_v1.3.docx"!

The document is now comprehensive enough that students can go from zero knowledge to successfully parsing GPS data with TinyGPSPlus. Would you like me to adjust anything?